Question 1: HTTP State Management, User Authentication, and Sessions

HTTP is a stateless protocol, which means that each request sent from a web browser to a web server is treated as an independent interaction. The server does not automatically remember information from previous requests.

To maintain the state of an application across multiple request-response cycles, web applications use techniques such as cookies, sessions, and authentication tokens. In Django, session management is commonly handled using the built-in session framework. When a user logs in, Django creates a session on the server and stores a unique session identifier in the user's browser as a cookie. During subsequent requests, the browser sends this cookie back to the server, allowing Django to identify the user and retrieve the associated session data.

This enables features such as user authentication, shopping carts, and personalized content. Django's authentication system integrates closely with sessions, allowing users to remain logged in as they navigate between pages without needing to re-enter their credentials. Security features such as session expiration, secure cookies, and protection against cross-site request forgery (CSRF) help safeguard user data and reduce the risk of unauthorized access. Through sessions and cookies, Django effectively maintains application state while still operating over the stateless HTTP protocol.

Question 2: Django Database Migrations to MariaDB

Django migrations provide a structured way to manage changes to a database schema over time. When deploying a Django application to a server-based relational database such as MariaDB, developers first configure the database connection settings in the project's settings.py file. This typically involves installing a MariaDB-compatible database driver, such as mysqlclient, and specifying connection details including the database name, username, password, host, and port.

After updating or creating Django models, the **python manage.py makemigrations** command is used to generate migration files that describe the required database changes. These migration files are then applied to the MariaDB database using the **python manage.py migrate** command.

Django records applied migrations in a special table called **django_migrations**, ensuring that schema updates are applied in the correct order and only once. Migrations can create new tables, modify existing columns, add indexes, and define relationships between tables. During deployment, running migrations helps keep the database schema synchronized with the application's models. This automated approach reduces manual database administration tasks, improves consistency across environments, and supports collaborative development by allowing schema changes to be tracked through version control systems such as Git.

References

1) Django Software Foundation. *How to Use Sessions*. Django Documentation

2) Mozilla Developer Network (MDN). *Django Tutorial Part 7: Sessions Framework*. MDN Web Docs

3) Django Software Foundation. *Migrations*. Django Documentation

4) MariaDB Corporation. *MariaDB Documentation*. MariaDB Documentation